

Homework 04 — STAT 432 (Fall 2026)

Name: Owen Park

NetID: owenp3

All four questions are done in Python (sklearn 1.6). Seeds 43246 (Q1, Q2) and 43247 (Q3) as the questions say. Covariates are centered and standardized with divisor n by hand, the response is centered, and the fitting functions get `fit_intercept=False` so nothing is rescaled inside them.

```
In [1]: import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import Lasso, ElasticNet, Ridge
pd.set_option("display.width", 120)
```

Question 1: Signal strength and correlated substitutes

$n=120$, $p=10$, $X_2 = 0.999 \cdot X_1 + \sqrt{1-0.999^2} \cdot Z$, $Y = 0.15X_1 + 0.15X_2 + 0.30X_3 + 0.15X_4 + \epsilon$.
200 reps, lasso at $\lambda \in \{0.08, 0.12, 0.18, 0.24\}$.

In sklearn the penalty goes in through `alpha` and there is no scaling option at all (Lasso never standardizes), so I standardize by hand with divisor n and pass

`fit_intercept=False . tol=1e-10, max_iter=100000` as required, fitted from large λ to small with `warm_start=True`. Slopes are divided by the learned scale to get back to the original units.

```
In [2]: def simulate(make_model):
    rng = np.random.default_rng(43246)
    n, p, R = 120, 10, 200
    beta = np.array([0.15, 0.15, 0.30, 0.15, 0, 0, 0, 0, 0, 0])
    lams = np.array([0.24, 0.18, 0.12, 0.08]) # decreasing, for warm
    B = np.zeros((R, p, 4)); iters = np.zeros((R, 4), int); corr = np.zeros(R)
    for k in range(R):
        U = rng.normal(size=(n, p))
        X = U.copy()
        X[:, 1] = 0.999 * U[:, 0] + np.sqrt(1 - 0.999**2) * U[:, 1]
        y = X @ beta + rng.normal(size=n)
        corr[k] = np.corrcoef(X[:, 0], X[:, 1])[0, 1]
        xbar = X.mean(0); Xc = X - xbar
        s = np.sqrt((Xc**2).mean(0)) # divisor n
        Xs = Xc / s; yc = y - y.mean()
        fit = make_model()
        for l, lam in enumerate(lams):
            fit.set_params(alpha=lam)
            fit.fit(Xs, yc)
            B[k, :, l] = fit.coef_ / s # back to original scale
```

```

        iters[k, l] = fit.n_iter_
    return lams, B, iters, corr

lams, B_lasso, it_lasso, corr = simulate(lambda: Lasso(fit_intercept=False, w
max_iter=100000, tol
print("all 800 fits converged:", bool((it_lasso < 100000).all()), "| max itera
print("sample corr(X1,X2): min %.4f mean %.4f max %.4f" % (corr.min(), corr.
sel_lasso = np.abs(B_lasso) > 1e-8
cols = [f"λ={l}" for l in lams]; idx = [f"X{j}" for j in range(1, 11)]
freq_lasso = pd.DataFrame(sel_lasso.mean(0), index=idx, columns=cols)
mag_lasso = pd.DataFrame(np.abs(B_lasso).mean(0), index=idx, columns=cols)
print("Selection frequency"); print(freq_lasso.round(3))
print("Mean |slope| (zeros included)"); print(mag_lasso.round(3))

```

all 800 fits converged: True | max iterations used: 10076

sample corr(X1,X2): min 0.9985 mean 0.9990 max 0.9994

Selection frequency

	λ=0.24	λ=0.18	λ=0.12	λ=0.08
X1	0.275	0.410	0.450	0.470
X2	0.425	0.510	0.545	0.555
X3	0.705	0.885	0.950	0.970
X4	0.155	0.330	0.615	0.775
X5	0.020	0.090	0.235	0.370
X6	0.010	0.060	0.205	0.385
X7	0.005	0.025	0.135	0.340
X8	0.025	0.065	0.170	0.375
X9	0.000	0.040	0.155	0.365
X10	0.005	0.065	0.215	0.385

Mean |slope| (zeros included)

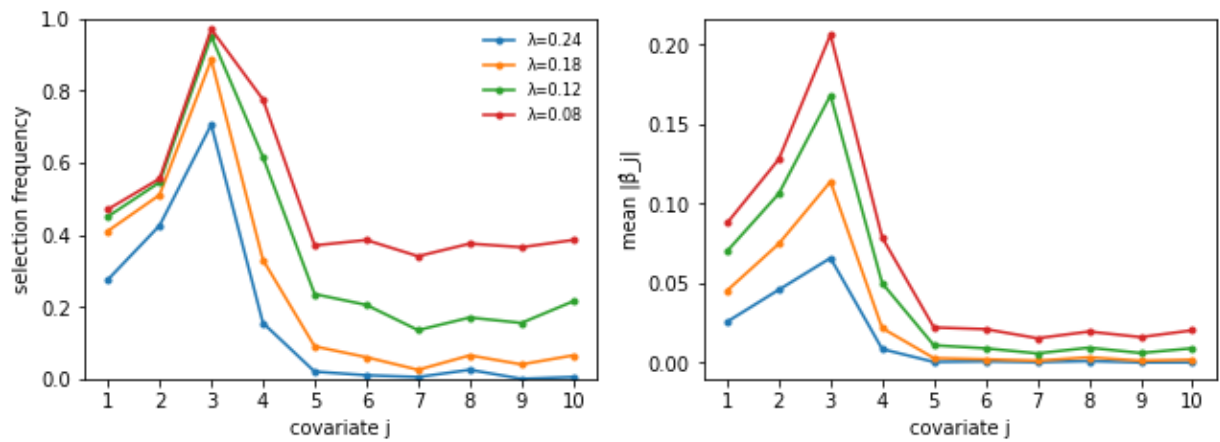
	λ=0.24	λ=0.18	λ=0.12	λ=0.08
X1	0.026	0.045	0.070	0.088
X2	0.046	0.075	0.106	0.128
X3	0.066	0.114	0.168	0.206
X4	0.008	0.022	0.050	0.079
X5	0.000	0.003	0.011	0.022
X6	0.001	0.002	0.009	0.021
X7	0.000	0.001	0.006	0.015
X8	0.001	0.003	0.009	0.019
X9	0.000	0.001	0.006	0.016
X10	0.000	0.002	0.009	0.020

1a

```

In [3]: fig, ax = plt.subplots(1, 2, figsize=(9, 3.4))
        for l, lam in enumerate(lams):
            ax[0].plot(range(1, 11), freq_lasso.iloc[:, l], marker="o", ms=3, label=f'
            ax[1].plot(range(1, 11), mag_lasso.iloc[:, l], marker="o", ms=3, label=f'
ax[0].set(xlabel="covariate j", ylabel="selection frequency", ylim=(0, 1), xti
ax[1].set(xlabel="covariate j", ylabel="mean |β̂_j|", xticks=range(1, 11))
ax[0].legend(frameon=False, fontsize=8); fig.tight_layout(); plt.show()

```



Both summaries go down as λ goes up, for every covariate. At $\lambda=0.08$ the noise variables X5–X10 are picked about 37% of the time, at $\lambda=0.24$ about 1%. X3 (true 0.30) goes $0.97 \rightarrow 0.705$, X4 (true 0.15) goes $0.775 \rightarrow 0.155$. The mean $|\text{slope}|$ shrinks the same way, X3 from 0.206 down to 0.066. This is what soft-thresholding does: each coordinate score gets λ subtracted, so a bigger λ both kills more small scores (lower selection) and pulls the survivors closer to zero (smaller magnitude).

1b

```
In [4]: def pair_table(sel, lams):
rows = []
for l, lam in enumerate(lams):
    a, b = sel[:, 0, l], sel[:, 1, l]
    rows.append({"λ": lam, "X1 only": (a & ~b).mean(), "X2 only": (~a & b).mean(),
                "both": (a & b).mean(), "neither": (~a & ~b).mean()})
return pd.DataFrame(rows)
pairs_lasso = pair_table(sel_lasso, lams)
print(pairs_lasso.round(3).to_string(index=False))
```

λ	X1 only	X2 only	both	neither
0.24	0.27	0.420	0.005	0.305
0.18	0.40	0.500	0.010	0.090
0.12	0.43	0.525	0.020	0.025
0.08	0.44	0.525	0.030	0.005

The two marginal frequencies are close (X1 0.45 vs X2 0.545 at $\lambda=0.12$) because the setup is symmetric: same true coefficient, same distribution, and they are almost the same column. Which one the lasso keeps is basically a coin flip decided by the noise in that dataset. The gap between them is within Monte Carlo error for 200 reps (SE of the difference is about $\sqrt{(0.43+0.525)/200} \approx 0.07$).

As λ goes down, "neither" disappears ($0.305 \rightarrow 0.005$) and almost everything becomes "one of them". "Both" stays tiny at every λ (0.005 to 0.03). Once one member is in the model, the partial residual the other one sees has almost no signal left in it, because the first one already explains the shared direction. Its score sits inside $[-\lambda, \lambda]$, so it gets zeroed. That is how the lasso ends up with a sparse rule that drops X1 or X2 even though both are truly nonzero: the dropped one is redundant given the kept one, not unimportant.

1c

```
In [5]: cmp_lasso = pd.DataFrame({"λ": lams, "X3": freq_lasso.iloc[2].values, "X4": fr
      "mean X5..X10": freq_lasso.iloc[4:10].mean().values])
print(cmp_lasso.round(3).to_string(index=False))
```

```

λ      X3      X4  mean X5..X10
0.24 0.705 0.155      0.011
0.18 0.885 0.330      0.057
0.12 0.950 0.615      0.186
0.08 0.970 0.775      0.370
```

X3 is above X4, and both are above the noise average, at every λ . At $\lambda=0.12$: 0.95, 0.615, 0.186. For an independent covariate the score is roughly the true coefficient plus noise with spread about $1/\sqrt{n} \approx 0.09$, so a coefficient of 0.30 clears the threshold λ most of the time, 0.15 only sometimes, and 0 only by chance. Raising λ widens the dead zone $[-\lambda, \lambda]$ and all three frequencies fall, the weak ones fastest.

The pair shows both warnings. A fitted zero does not mean the true coefficient is zero: X1 and X2 both have true coefficient 0.15 and each one is zeroed about half the time. A nonzero fit does not mean the variable is uniquely important either: the kept member is standing in for the pair, and in the next dataset it is the other one. What the lasso gives is one sparse rule that predicts well, not the generating equation.

Question 2: Elastic net with an equal penalty mix

Same data (same seed, same draws), `ElasticNet(l1_ratio=0.5)`. In sklearn the mix is `l1_ratio` and the total penalty is `alpha`; in glmnet it is the other way round (`alpha` is the mix). Everything else is as in Q1.

```
In [6]: _, B_en, it_en, _ = simulate(lambda: ElasticNet(fit_intercept=False, warm_star
      max_iter=100000, tol=1e-10,
print("all 800 fits converged:", bool((it_en < 100000).all()), "| max iteratio
sel_en = np.abs(B_en) > 1e-8
freq_en = pd.DataFrame(sel_en.mean(0), index=idx, columns=cols)
print("Selection frequency (elastic net)"); print(freq_en.round(3))
print("Change vs lasso"); print((freq_en - freq_lasso).round(3))
```

all 800 fits converged: True | max iterations used: 236

Selection frequency (elastic net)

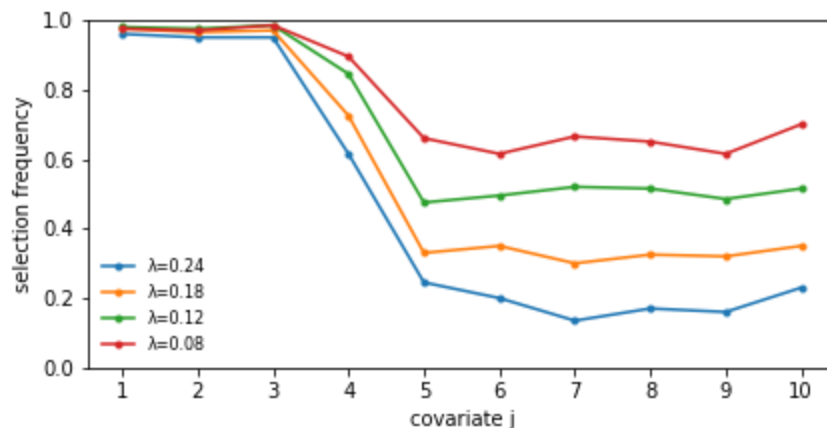
	$\lambda=0.24$	$\lambda=0.18$	$\lambda=0.12$	$\lambda=0.08$
X1	0.960	0.975	0.980	0.975
X2	0.950	0.965	0.975	0.970
X3	0.950	0.970	0.985	0.985
X4	0.615	0.725	0.845	0.895
X5	0.245	0.330	0.475	0.660
X6	0.200	0.350	0.495	0.615
X7	0.135	0.300	0.520	0.665
X8	0.170	0.325	0.515	0.650
X9	0.160	0.320	0.485	0.615
X10	0.230	0.350	0.515	0.700

Change vs lasso

	$\lambda=0.24$	$\lambda=0.18$	$\lambda=0.12$	$\lambda=0.08$
X1	0.685	0.565	0.530	0.505
X2	0.525	0.455	0.430	0.415
X3	0.245	0.085	0.035	0.015
X4	0.460	0.395	0.230	0.120
X5	0.225	0.240	0.240	0.290
X6	0.190	0.290	0.290	0.230
X7	0.130	0.275	0.385	0.325
X8	0.145	0.260	0.345	0.275
X9	0.160	0.280	0.330	0.250
X10	0.225	0.285	0.300	0.315

2a

```
In [7]: fig, ax = plt.subplots(figsize=(6, 3.2))
for l, lam in enumerate(lams):
    ax.plot(range(1, 11), freq_en.iloc[:, l], marker="o", ms=3, label=f" $\lambda={lam}$ ")
ax.set(xlabel="covariate j", ylabel="selection frequency", ylim=(0, 1), xticks=range(1, 11))
ax.legend(frameon=False, fontsize=8); fig.tight_layout(); plt.show()
```



Every frequency goes up compared to the lasso. The biggest jumps are the correlated pair: at $\lambda=0.12$ X1 goes $0.45 \rightarrow 0.98$ and X2 $0.545 \rightarrow 0.975$ (about +50 points each). X3 is already near 1 in both. X4 goes $0.615 \rightarrow 0.845$. The noise covariates also go up a lot, from 0.186 to 0.50 on average, which is the price.

2b

```
In [8]: pairs_en = pair_table(sel_en, lams)
print("elastic net"); print(pairs_en.round(3).to_string(index=False))
```

```
print("lasso (Q1)"); print(pairs_lasso.round(3).to_string(index=False))
```

```
elastic net
  λ  X1 only  X2 only  both  neither
0.24  0.025  0.015  0.935  0.025
0.18  0.020  0.010  0.955  0.015
0.12  0.020  0.015  0.960  0.005
0.08  0.025  0.020  0.950  0.005
lasso (Q1)
  λ  X1 only  X2 only  both  neither
0.24  0.27  0.420  0.005  0.305
0.18  0.40  0.500  0.010  0.090
0.12  0.43  0.525  0.020  0.025
0.08  0.44  0.525  0.030  0.005
```

Now "both" is 0.935 to 0.96 at every λ , where the lasso had 0.005 to 0.03. Single-member fits almost vanish (about 2% each side) and "neither" is at most 2.5%. So yes, the pair is now selected together.

Why: for two almost identical columns, splitting the shared effect any way between β_1 and β_2 gives nearly the same fitted values and the same $|\beta_1| + |\beta_2|$, so the ℓ_1 part does not care how the sum is split, and the lasso just keeps whichever wins the tie. The squared part $\beta_1^2 + \beta_2^2$ does care: for a fixed sum it is smallest when the two are equal. That strictly convex piece breaks the tie in favor of sharing, so both stay in.

2c

```
In [9]: cmp_en = pd.DataFrame({"λ": lams, "noise (enet)": freq_en.iloc[4:10].mean().values,
                             "noise (lasso)": freq_lasso.iloc[4:10].mean().values,
                             "pair both (enet)": pairs_en["both"].values})
print(cmp_en.round(3).to_string(index=False))
```

```
  λ  noise (enet)  noise (lasso)  pair both (enet)
0.24          0.190          0.011          0.935
0.18          0.329          0.057          0.955
0.12          0.501          0.186          0.960
0.08          0.651          0.370          0.950
```

The noise covariates still get screened out more as λ grows (0.65 $\rightarrow$ 0.19) but much less than with the lasso at the same number (0.37 $\rightarrow$ 0.01). With $l1_ratio$ 0.5, the soft-threshold applied to a coordinate is $\lambda \cdot 0.5$, half the lasso's, and the ℓ_2 half of the penalty never makes anything exactly zero. So at the same λ the elastic net is a weaker screener and needs a bigger λ to match the lasso.

On this grid, $\lambda=0.24$ is the one where both goals are closest to met: the pair is kept together 93.5% of the time and the noise average is at its lowest, 0.19. Even there, one in five noise variables sneaks in, so screening as hard as the lasso would need a λ above the grid.

Question 3: Coordinate descent and the lasso path

$n=100$, $p=3$, $Y = 1.5 + X_1 - 2X_2 + \epsilon$, seed 43247. Response centered, covariates standardized with divisor n so $(1/n)x_j^T x_j = 1$ and the coordinate update is exactly soft-

thresholding of the score a_j . Warm starts down the grid.

```
In [10]: def soft(a, lam):
         return np.sign(a) * max(abs(a) - lam, 0.0)

def lasso_path(X, y, grid, tol=1e-8, max_iter=1000):
    beta = np.zeros(X.shape[1]); path = []; cycles = []
    for lam in grid:
        for t in range(max_iter):
            old = beta.copy()
            for j in range(X.shape[1]):
                r_j = y - X @ beta + X[:, j] * beta[j]      # residual without
                a_j = np.mean(X[:, j] * r_j)                 # score
                beta[j] = soft(a_j, lam)
            if np.max(np.abs(beta - old)) < tol:
                break
        else:
            raise RuntimeError(f"no convergence at lambda={lam}")
        path.append(beta.copy()); cycles.append(t + 1)
    return np.array(path), np.array(cycles)

rng = np.random.default_rng(43247)
n = 100
X = rng.normal(size=(n, 3))
y = 1.5 + X[:, 0] - 2 * X[:, 1] + rng.normal(size=n)
xbar = X.mean(0); s = np.sqrt(((X - xbar)**2).mean(0)); Xs = (X - xbar) / s
yc = y - y.mean()

grid = np.array([2.5, 2.0, 1.5, 1.0, 0.6, 0.4, 0.25, 0.15, 0.08, 0.03])
path, cycles = lasso_path(Xs, yc, grid)
tab = pd.DataFrame(path, columns=[" $\hat{\beta}_1$ ", " $\hat{\beta}_2$ ", " $\hat{\beta}_3$ "]); tab.insert(0, " $\lambda$ ", grid)
print(tab.round(4).to_string(index=False))
lam_max = np.abs((Xs * yc[:, None]).mean(0)).max()
print("intercept (mean of y) = %.4f" % y.mean())
print("lambda_max = %.4f  scores at beta=0:" % lam_max, np.round((Xs * yc[:,
ols = np.linalg.lstsq(Xs, yc, rcond=None)[0]
print("least squares on the same standardized data:", ols.round(4))
```

λ	$\hat{\beta}_1$	$\hat{\beta}_2$	$\hat{\beta}_3$	cycles
2.50	0.0000	-0.0000	-0.0000	1
2.00	0.0000	-0.2373	-0.0000	2
1.50	0.0000	-0.7373	-0.0000	2
1.00	0.0000	-1.2373	-0.0000	2
0.60	0.2553	-1.6183	-0.0000	5
0.40	0.4416	-1.8046	0.0000	5
0.25	0.5812	-1.9442	0.0000	5
0.15	0.6743	-2.0373	0.0000	5
0.08	0.7441	-2.1168	0.0587	7
0.03	0.7963	-2.1807	0.1298	7

intercept (mean of y) = 1.7762
lambda_max = 2.2373 scores at beta=0: [0.9752 -2.2373 -0.4628]
least squares on the same standardized data: [0.8276 -2.2191 0.1724]

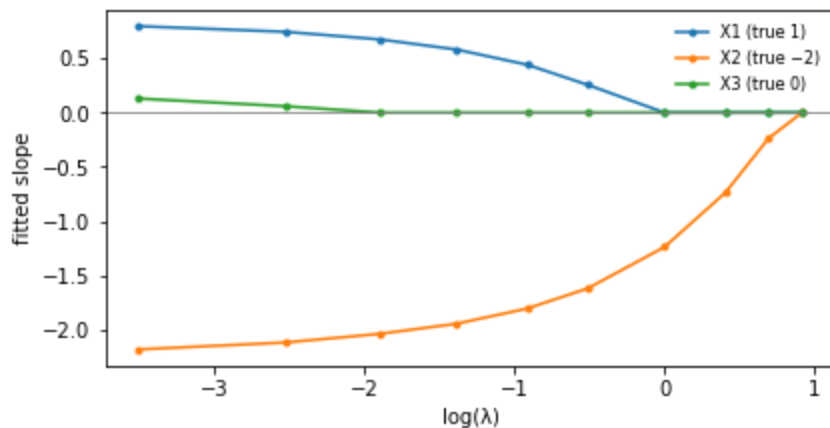
3a

Every fit converged (the function raises otherwise); the cycle counts are in the table, 1 for the cold start and at most 7 with warm starts. $\lambda_{\max} = 2.2373$, which is $|a_2|$ at $\beta=0$. The only grid value at or above it is 2.5, and that fit is all zeros. At 2.0, just below $\lambda_{\max}$, X_2

already comes in with -0.2373 , which is exactly $S(-2.2373, 2.0)$. So the threshold rule checks out.

3b

```
In [11]: fig, ax = plt.subplots(figsize=(6, 3.2))
for j, name in enumerate(["X1 (true 1)", "X2 (true -2)", "X3 (true 0)"]):
    ax.plot(np.log(grid), path[:, j], marker="o", ms=3, label=name)
ax.axhline(0, color="gray", lw=0.8); ax.set(xlabel="log( $\lambda$ )", ylabel="fitted slope")
ax.legend(frameon=False, fontsize=8); fig.tight_layout(); plt.show()
```



X2 enters first (at $\lambda=2.0$), X1 second (between 1.0 and 0.6), X3 last (at 0.08). The order follows the size of the true coefficients: $|-2| > |1| > 0$. With independent covariates the score a_j at $\beta=0$ is about the true coefficient, so the bigger it is the larger the λ at which it clears the threshold. As $\lambda \rightarrow 0$ both active slopes move toward their least-squares values.

3c

Yes, X3 enters at the two smallest penalties: 0.0587 at $\lambda=0.08$ and 0.1298 at $\lambda=0.03$. Its true coefficient is zero but in this sample its least-squares slope is 0.1724 and its partial-residual score at $\lambda=0.03$ is about 0.16, which is above the threshold. Whether a null variable gets in at small λ depends on the sample, so the path is not a ranking of importance.

At $\lambda=0.03$ the fit is (0.7963, -2.1807 , 0.1298) and least squares is (0.8276, -2.2191 , 0.1724). Close but not the same: the lasso solution at any $\lambda>0$ still has λ subtracted from each active score, so every active coefficient sits a bit closer to zero than OLS. With correlated sample columns that does not have to hold coordinate by coordinate, but here it does for all three. As $\lambda \rightarrow 0$ the penalty disappears and the two coincide.

Question 4: Tuning and comparing penalized regressions for diabetes prediction

442 rows, 353 train / 89 test, ten folds given in the file. `GridSearchCV` with 61 log-spaced `alpha` values from $1e-4$ to $1e2$, `StandardScaler` inside a `Pipeline` so every fold re-estimates the centers and scales from its own training rows. The penalty family is

the estimator class (`Lasso` , `Ridge` , `ElasticNet(l1_ratio=0.5)`); `alpha` is the strength. Intercept is fitted, not penalized (sklearn centers y internally).

```
In [12]: from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

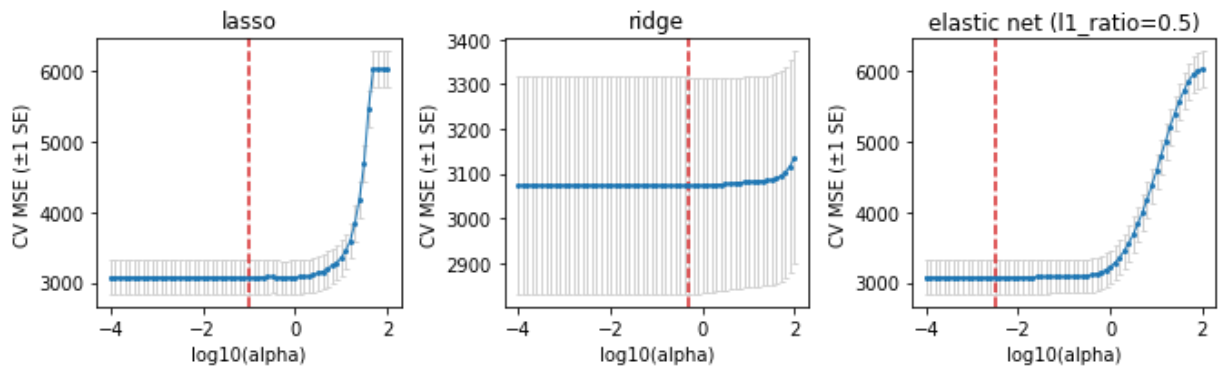
diab = pd.read_csv("data/diabetes.csv"); diab.insert(0, "row_id", np.arange(1,
folds = pd.read_csv("data/diabetes-split-folds.csv")
dat = folds.merge(diab, on="row_id", how="left", validate="one_to_one")
feats = ["age", "sex", "bmi", "bp", "s1", "s2", "s3", "s4", "s5", "s6"]
is_tr = dat["split"].eq("train").to_numpy()
Xall = dat[feats].to_numpy(float); yall = dat["y"].to_numpy(float)
Xtr, ytr, Xte, yte = Xall[is_tr], yall[is_tr], Xall[~is_tr], yall[~is_tr]
fold = dat.loc[is_tr, "cv_fold"].to_numpy(int)
splits = [(np.flatnonzero(fold != k), np.flatnonzero(fold == k)) for k in np.splits
print("train", len(ytr), "test", len(yte), "folds", len(splits))

alphas = np.logspace(-4, 2, 61)
specs = {"lasso": Lasso(max_iter=100000, tol=1e-10), "ridge": Ridge(),
        "elastic net (l1_ratio=0.5)": ElasticNet(l1_ratio=0.5, max_iter=100000)}
searches, rows = {}, []
for name, est in specs.items():
    gs = GridSearchCV(Pipeline([("scale", StandardScaler()), ("model", est)]),
                      {"model__alpha": alphas}, scoring="neg_mean_squared_error")
    gs.fit(Xtr, ytr); searches[name] = gs; b = gs.best_index_
    rows.append({"method": name, "alpha_min": gs.best_params_["model__alpha"],
                "cv_mse": -gs.cv_results_["mean_test_score"][b],
                "cv_se": gs.cv_results_["std_test_score"][b] / np.sqrt(len(splits))})
summary = pd.DataFrame(rows); print(summary.round(4).to_string(index=False))

train 353 test 89 folds 10
      method  alpha_min  cv_mse  cv_se
      lasso      0.1000 3071.2469 240.3527
      ridge      0.5012 3073.4858 242.1468
elastic net (l1_ratio=0.5)  0.0032 3073.4487 242.1001
```

```
In [13]: fig, axes = plt.subplots(1, 3, figsize=(9.5, 3.0))
for ax, (name, gs) in zip(axes, searches.items()):
    a = gs.cv_results_["param_model__alpha"].data.astype(float)
    m = -gs.cv_results_["mean_test_score"]; se = gs.cv_results_["std_test_score"]
    ax.errorbar(np.log10(a), m, yerr=se, marker="o", ms=2, lw=1, capsize=1.5,
    ax.axvline(np.log10(gs.best_params_["model__alpha"]), color="C3", ls="--")
    ax.set(title=name, xlabel="log10(alpha)", ylabel="CV MSE (±1 SE)")
fig.tight_layout(); plt.show()

winner = summary.loc[summary["cv_mse"].idxmin(), "method"]
best = searches[winner].best_estimator_
coef_raw = best.named_steps["model"].coef_ / best.named_steps["scale"].scale_
print("selected:", winner, "| alpha =", searches[winner].best_params_["model__alpha"])
print("nonzero coefficients:", int((np.abs(coef_raw) > 1e-8).sum()), "of 10; coefficients are")
    [f for f, c in zip(feats, coef_raw) if abs(c) > 1e-8])
print("training MSE = %.1f" % np.mean((ytr - best.predict(Xtr))**2))
print("final TEST MSE (evaluated once) = %.1f" % np.mean((yte - best.predict(Xte))**2))
```



selected: lasso | $\alpha = 0.1$
 nonzero coefficients: 9 of 10; dropped: ['s3']
 training MSE = 2908.4
 final TEST MSE (evaluated once) = 2732.4

The three tuned models are basically tied: CV MSE 3071.2 (lasso, $\alpha=0.1$), 3073.5 (ridge, $\alpha \approx 0.50$), 3073.4 (elastic net, $\alpha \approx 0.0032$), with fold SEs around 240. The differences are tiny next to the fold-to-fold spread, so this does not say one method is really better, it just picks a nominal winner for this fold split. The plots show the same thing: a long flat bottom for all three, with the dashed line at the minimum.

Selected: **lasso, $\alpha = 0.1$** . Refit on all 353 training rows it keeps 9 of 10 covariates (drops `s3`), training MSE 2908.4, and the one-time test MSE on the 89 held-out rows is **2732.4**. The test error came out a bit below the CV error, which can go either way on another split. The test rows were not touched until the method and penalty were fixed, so that number is an honest estimate of the whole procedure.

Note the alphas are not comparable across methods, since each objective is normalized differently (elastic net's 0.0032 is not "less penalty" than the lasso's 0.1 in any useful sense); only the CV errors on the same folds are.